

Python: Syntax & Kontrollstrukturen

Kurze Beispiele. Kommentare (# ...) nur dort, wo sich Python von Java unterscheidet.

1. Syntax-Grundlagen

Blöcke & Einrückung

Java: { } · Python: Einrückung + Doppelpunkt nach if/for/def/class/try

```
x = 10
if x > 0:
    print("positiv")
    x -= 1
print("fertig")
```

Variablen & Typen

Java: int x = 10; · Python: dynamisch typisiert, kein Typ-Keyword nötig

```
name = "Ada"
alter = 36
pi = 3.14
aktiv = True           # Java: true/false (klein) · Python: True/False
nichts = None          # Java: null · Python: None
```

Vergleiche & logische Operatoren

Java: && || ! · Python: and / or / not · Gleichheit: == · Identität: is

```
a, b = 5, 10
print(a < b and b != 0)
print(not (a == b))
print(a is not None)
```

2. if / elif / else

Java: else if · Python: elif · Bedingungen ohne Klammern (optional)

```
note = 78

if note >= 90:
    print("sehr gut")
elif note >= 75:
    print("gut")
elif note >= 50:
    print("genügend")
else:
    print("nicht genügend")

# Einzeiler (selten nötig):
status = "ok" if note >= 50 else "fail"    # Java: ternär ? :
```

3. Schleifen

for

Java: for (int i=0; i<n; i++) · Python: for über Iterables / range

```
for i in range(5):           # 0..4
    print(i)

for i in range(2, 10, 2):    # Start, Stop (exklusiv), Schritt
    print(i)

namen = ["Ana", "Ben", "Cara"]
for name in namen:
    print(name)

for index, name in enumerate(namen):
    print(index, name)
```

while

```
n = 3
while n > 0:
    print(n)
    n -= 1                # Java: n-- / n++ · Python: kein ++/--
```

4. break, continue, pass

break / continue wie in Java · pass: leerer Block (Java hat dafür { })

```
for i in range(10):
    if i == 3:
        continue
    if i == 7:
        break
    print(i)
```

```
x = 1
if x > 0:
    pass
```

```
def noch_offen():
    pass
```

else an Schleifen

Java: kein Schleifen-else · Python: else läuft, wenn die Schleife nicht per break endet

```
for n in range(2, 10):
    for teiler in range(2, n):
        if n % teiler == 0:
            break
    else:
        print(n, "ist prim")
```

5. try / except / else / finally

Java: try/catch/finally · Python: except · mehrere Typen: except (A, B)

```
def teile(a, b):
    try:
        ergebnis = a / b
    except ZeroDivisionError:
        print("Division durch 0")
        return None
    except TypeError as e:          # Java: catch (TypeError e)
        print("Typfehler:", e)
        return None
    else:                          # nur ohne Exception (kein Java-Äquivalent)
        print("ok")
        return ergebnis
    finally:
        print("fertig")
```

```
print(teile(10, 2))
print(teile(10, 0))
```

Exceptions auslösen: raise ValueError(...) · Java: throw new ...

```
def alter_setzen(alter):
    if alter < 0:
        raise ValueError("Alter darf nicht negativ sein")
    return alter
```

Kurz: Python vs. Java

- Blöcke über Einrückung, nicht über { }
- elif statt else if; True/False/None statt true/false/null
- and / or / not statt && / || / !
- for über Sequenzen / range, kein C-Style-for
- kein ++ / --; pass als leerer Block
- except statt catch; optional else am try und an Schleifen